

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**



**BÁO CÁO ĐỒ ÁN MÔN HỌC**

**BIỂU DIỄN TRI THỨC VÀ ỨNG DỤNG**

**ỨNG DỤNG MÔ HÌNH ONTOLOGY COKB  
TRONG ĐÁNH GIÁ VÀ GIẢI THÍCH  
ÁNH XẠ MÃ Y SINH**

Giảng viên hướng dẫn: PGS.TS. Đỗ Văn Nhon

Học viên thực hiện:  
1. Trần Hoàng Anh Khoa – 25C01037  
2. Hồ Võ Hoàng Duy – 25C01002

Thành phố Hồ Chí Minh, tháng 9 năm 2026

# Tóm tắt

Báo cáo trình bày việc vận dụng mô hình Ontology COKB để tổ chức, đánh giá và giải thích dữ liệu ánh xạ giữa các hệ thống mã y sinh. Bài toán đặt ra là: với một cặp mã nguồn–đích đã có, hệ thống có thể đưa ra nhận định nào, dựa trên điều kiện nào và phải để lại bằng chứng gì để người dùng kiểm tra? Đề án xây dựng lớp tri thức gồm ontology, ràng buộc dữ liệu, toán tử xử lý nhãn, năm luật đánh giá có thứ tự ưu tiên và vết suy luận có cấu trúc.

Phần cài đặt sử dụng Python và RDF/Turtle; SHACL được cung cấp như một bước kiểm tra bổ sung. Dữ kiện nguồn, kết quả đánh giá và vết suy luận được xuất thành ba nhóm dữ liệu riêng. Bộ suy luận trả về A, B, C hoặc UNASSESSED; trạng thái cuối thể hiện rằng tập luật chưa xác lập được kết luận. Đây là một vận dụng COKB trong miền hẹp, với luật xử lý nhãn được hiện thực bằng Python, chưa phải bộ giải đối tượng tính toán tổng quát.

Thực nghiệm tái lập trên 500 cặp nhãn đạt độ bao phủ 4,40% và độ chính xác 95,45% trong 22 cặp được đánh giá; độ chính xác tính trên toàn bộ tập là 4,20%. Kết quả cho thấy cơ chế suy luận và truy vết hoạt động, đồng thời chỉ ra giới hạn lớn về độ bao phủ. Giá trị thực tế của đề án nằm ở khả năng hỗ trợ rà soát ánh xạ một cách nhất quán, chỉ rõ nguồn gốc kết luận và nhận diện các trường hợp cần chuyên gia xem xét.

**Từ khóa:** COKB, biểu diễn tri thức, ánh xạ mã y sinh, luật suy luận, RDF, SHACL, vết suy luận.

# Mục lục

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Tóm tắt</b>                                         | <b>i</b>  |
| <b>Danh mục thuật ngữ</b>                              | <b>vi</b> |
| <b>1 Bài toán và ý nghĩa thực tế</b>                   | <b>1</b>  |
| 1.1 Động cơ thực hiện . . . . .                        | 1         |
| 1.2 Phát biểu bài toán . . . . .                       | 1         |
| 1.3 COKB mang lại giá trị gì? . . . . .                | 1         |
| 1.4 Mục tiêu và phạm vi . . . . .                      | 2         |
| <b>2 Mô hình biểu diễn tri thức</b>                    | <b>3</b>  |
| 2.1 Vận dụng mô hình COKB . . . . .                    | 3         |
| 2.2 Các đối tượng và quan hệ . . . . .                 | 4         |
| 2.3 Phân cấp và mức độ hoàn thiện . . . . .            | 4         |
| 2.4 Dữ kiện nguồn, kết luận và bằng chứng . . . . .    | 5         |
| 2.5 Ràng buộc tính hợp lệ . . . . .                    | 5         |
| <b>3 Toán tử và cơ chế suy luận</b>                    | <b>6</b>  |
| 3.1 Chuẩn hóa và trích đặc trưng . . . . .             | 6         |
| 3.2 Phát hiện xung đột theo cụm từ . . . . .           | 6         |
| 3.3 Năm luật và thứ tự áp dụng . . . . .               | 7         |
| 3.4 Vết suy luận và ý nghĩa của tính tái lập . . . . . | 8         |
| 3.5 Độ phức tạp và giới hạn biểu diễn . . . . .        | 8         |
| <b>4 Thiết kế và cài đặt hệ thống</b>                  | <b>9</b>  |
| 4.1 Kiến trúc xử lý . . . . .                          | 9         |
| 4.2 Phân rã module . . . . .                           | 9         |
| 4.3 Định danh và nhập dữ liệu . . . . .                | 10        |
| 4.4 Xử lý lỗi và các artifact . . . . .                | 10        |
| 4.5 Ví dụ giải thích một mapping . . . . .             | 11        |
| <b>5 Thực nghiệm và phân tích kết quả</b>              | <b>12</b> |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 5.1      | Thiết kế thực nghiệm . . . . .                 | 12        |
| 5.2      | Các chỉ số . . . . .                           | 13        |
| 5.3      | Kết quả chức năng trên năm mapping . . . . .   | 13        |
| 5.4      | Kết quả trên bộ gold . . . . .                 | 14        |
| 5.5      | Mức đóng góp của từng luật . . . . .           | 15        |
| 5.6      | Phân tích lỗi cụ thể . . . . .                 | 16        |
| 5.7      | Kiểm thử và tính tái lập . . . . .             | 16        |
| <b>6</b> | <b>Thảo luận, kết luận và hướng phát triển</b> | <b>17</b> |
| 6.1      | Đóng góp của đồ án . . . . .                   | 17        |
| 6.2      | Ý nghĩa trong một quy trình sử dụng . . . . .  | 17        |
| 6.3      | Giới hạn cần nhìn nhận . . . . .               | 17        |
| 6.4      | Hướng phát triển ưu tiên . . . . .             | 18        |
| 6.5      | Kết luận . . . . .                             | 18        |
| <b>A</b> | <b>Hướng dẫn tái lập và đối chiếu source</b>   | <b>19</b> |
| A.1      | Các lệnh tái lập . . . . .                     | 19        |
| A.2      | Chỉ mục đối chiếu nội dung với code . . . . .  | 20        |
| A.3      | Danh sách hình cần bổ sung . . . . .           | 20        |
|          | <b>Tài liệu tham khảo</b>                      | <b>22</b> |

# Danh mục hình

|     |                                                                                                                                |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Các đối tượng trung tâm và chiều quan hệ được xuất trong RDF. . . . .                                                          | 4  |
| 4.1 | Luồng COKB được thực thi bởi các lệnh CLI hiện tại. . . . .                                                                    | 9  |
| 4.2 | Kết quả truy vấn COKB cho mã nguồn K05.1. . . . .                                                                              | 11 |
| 4.3 | Trực quan hóa vết suy luận cho mapping K05.1 đến DA0B.Y. . . . .                                                               | 11 |
| 5.1 | Phân bố kết quả theo nhãn chuẩn. Phần chưa đánh giá chiếm ưu thế ở cả ba nhóm; các số đếm chi tiết nằm trong Bảng 5.4. . . . . | 15 |
| 5.2 | Minh họa kết quả kiểm thử và đánh giá tái lập. . . . .                                                                         | 16 |

# Danh mục bảng

|     |                                                                      |    |
|-----|----------------------------------------------------------------------|----|
| 1.1 | Ý nghĩa đầu ra và cách sử dụng dự kiến . . . . .                     | 2  |
| 2.1 | Đối chiếu sáu thành phần COKB với source code . . . . .              | 3  |
| 3.1 | Mã luật và bằng chứng được trả về khi luật khớp . . . . .            | 7  |
| 4.1 | Trách nhiệm các module của lớp COKB . . . . .                        | 10 |
| 5.1 | Các dữ liệu và kiểm tra được sử dụng . . . . .                       | 12 |
| 5.2 | Đầu ra tái lập trên fixture có sẵn . . . . .                         | 13 |
| 5.3 | Kết quả tổng hợp của bộ suy luận xác định . . . . .                  | 14 |
| 5.4 | Ma trận nhầm lẫn: hàng là gold, cột là dự đoán . . . . .             | 14 |
| 5.5 | Precision, recall và F1 theo từng mức; đơn vị % . . . . .            | 14 |
| 5.6 | Số lần từng luật được chọn trên 500 cặp gold . . . . .               | 15 |
| 6.1 | Khoảng cách giữa hiện thực hiện tại và hệ thống hoàn chỉnh . . . . . | 18 |
| A.1 | Các điểm kiểm chứng chính trong repository . . . . .                 | 20 |

## Danh mục thuật ngữ

| Thuật ngữ   | Diễn giải trong báo cáo                                                          |
|-------------|----------------------------------------------------------------------------------|
| COKB        | Computational Object Knowledge Base: mô hình cơ sở tri thức đối tượng tính toán. |
| ICD         | International Classification of Diseases: hệ thống phân loại bệnh.               |
| RDF         | Resource Description Framework: biểu diễn dữ liệu bằng bộ ba.                    |
| SHACL       | Shapes Constraint Language: mô tả và kiểm tra ràng buộc trên graph RDF.          |
| Assertion   | Phát biểu ánh xạ được nạp từ nguồn, chưa đồng nghĩa với ánh xạ đúng.             |
| Assessment  | Kết quả đánh giá một phát biểu ánh xạ.                                           |
| Proof trace | Vết suy luận gồm kết luận, tiền đề, mã luật và nguồn bằng chứng.                 |
| Coverage    | Tỷ lệ cặp nhãn được gán A, B hoặc C.                                             |
| Abstention  | Việc hệ thống trả về UNASSESSED thay vì gán A/B/C.                               |

# Chương 1

## Bài toán và ý nghĩa thực tế

### 1.1 Động cơ thực hiện

Trong tích hợp dữ liệu, cùng một nội dung bệnh học có thể được mô tả bằng các hệ thống mã khác nhau. Một bảng ánh xạ cung cấp các cặp mã ứng viên, nhưng việc có mặt trong bảng chưa giải quyết được câu hỏi về mức độ phù hợp của từng cặp. Người kiểm tra cần biết nhân có tương ứng hay không, có khác mức độ chi tiết hay có từ ngữ chỉ tình trạng xung đột. Nhu cầu hỗ trợ chuyên gia kiểm tra các ứng viên ánh xạ cũng là bối cảnh của công trình OntologyRAG [2].

Đồ án sử dụng mã nguồn và dữ liệu hiện có trong repository làm nền tảng, tập trung vào lớp tri thức COKB. Vấn đề cốt lõi là xây dựng một quy trình *đánh giá có căn cứ*: cùng dữ kiện và cùng phiên bản luật phải cho cùng nhận định, đồng thời cho phép truy ngược nhận định đó đến tiền đề và nguồn.

### 1.2 Phát biểu bài toán

Mỗi ứng viên ánh xạ được mô hình hóa bởi

$$m = (id, c_s, c_t, e, b), \quad c = (s, k, \ell), \quad (1.1)$$

trong đó  $id$  là định danh;  $c_s, c_t$  là khái niệm nguồn và đích;  $s$  là tên hệ mã;  $k$  là giá trị mã;  $\ell$  là nhân;  $e$  là nguồn bằng chứng;  $b$  là tác nhân khai báo. Đầu ra của hệ thống là

$$f_{\mathcal{K}}(m) = (y, \tau, \pi), \quad y \in \{A, B, C, \perp\}, \quad \tau = \text{rule\_based}, \quad (1.2)$$

với  $\perp$  tương ứng UNASSESSED và  $\pi$  là vết suy luận. Tập tri thức  $\mathcal{K}$  bao gồm mô hình dữ liệu, các phép xử lý và tập luật được chọn cho lần chạy. Việc tìm ra mã đích mới ngoài các ứng viên đầu vào không nằm trong hàm  $f_{\mathcal{K}}$ .

Các hành động nghiệp vụ trong Bảng 1.1 là cách sử dụng được đề xuất. Code hiện trả assessment và proof; chưa có bộ chính sách phê duyệt, hàng đợi chuyên gia hay chức năng tự động chuyển dữ liệu bệnh viện. Đặc biệt, A là kết quả so sánh nhân theo thuật toán, không phải chứng nhận tương đương y khoa giữa hai khái niệm.

### 1.3 COKB mang lại giá trị gì?

Giá trị thứ nhất là **biến tiêu chí kiểm tra thành tri thức có thể thực thi**. Ví dụ, tiêu chí “hai nhân trùng nhau sau chuẩn hóa” được đóng gói thành một luật có định danh. Khi tiêu chí thay đổi, người phát triển biết phải sửa thành phần nào và đo lại ảnh hưởng lên dữ liệu nào.



**Bảng 1.1:** Ý nghĩa đầu ra và cách sử dụng dự kiến

| Đầu ra | Diễn giải theo bộ luật hiện tại                              | Hỗ trợ công việc nào?                              |
|--------|--------------------------------------------------------------|----------------------------------------------------|
| A      | Nhãn trùng nhau sau chuẩn hóa.                               | Đưa các cặp để kiểm tra vào nhóm đề xuất xác nhận. |
| B      | Thỏa điều kiện bao hàm từ hoặc khái quát hóa theo heuristic. | Chú ý khác biệt về nhãn; kiểm tra thêm ngữ cảnh.   |
| C      | Phát hiện cặp từ/cụm từ đối lập đã khai báo.                 | Đưa vào nhóm cảnh báo xung đột.                    |
| ⊥      | Không có luật A/B/C nào khớp.                                | Giữ lại cho chuyên gia hoặc bổ sung bằng chứng.    |

Giá trị thứ hai là **giải trình được một quyết định cụ thể**. Với một cảnh báo xung đột, hệ thống cung cấp cả hai nhãn và danh sách cụm từ đã kích hoạt luật. Chuyên gia có thể nhận ra cảnh báo đúng hoặc phát hiện lỗi do thuật toán hiểu sai phủ định. Đây là lợi ích trực tiếp của proof trace đối với kiểm tra chất lượng dữ liệu.

Giá trị thứ ba là **biểu diễn rõ giới hạn hiểu biết**. UNASSESSED giữ lại những cặp mà luật chưa xử lý được. Nếu được tích hợp vào quy trình rà soát, trạng thái này giúp hình thành danh sách cần kiểm tra và giúp nhóm phát triển ưu tiên bổ sung tri thức. Tác dụng tiết kiệm thời gian cho chuyên gia vẫn cần được đo bằng nghiên cứu người dùng; đồ án chưa có thực nghiệm xác nhận lợi ích định lượng đó.

## 1.4 Mục tiêu và phạm vi

Đồ án hướng đến bốn kết quả: mô hình hóa các đối tượng và quan hệ của ánh xạ; hiện thực các phép kiểm tra cùng luật đánh giá; xuất được kết luận kèm vết suy luận; và đánh giá chất lượng bằng dữ liệu có nhãn tham chiếu. Các ví dụ ICD chỉ được diễn giải theo nhãn có trong repository.

Phạm vi thực nghiệm gồm năm mapping mẫu và 500 cặp nhãn gold. Hệ thống chưa xử lý đầy đủ ngữ nghĩa phân cấp bệnh, phiên bản ICD, ngữ cảnh lâm sàng hoặc ánh xạ đích dạng tổ hợp. Báo cáo phân biệt các khả năng đã chạy được, các khái niệm chỉ mới khai báo trong ontology và các hướng ứng dụng cần triển khai thêm.

## Chương 2

# Mô hình biểu diễn tri thức

### 2.1 Vận dụng mô hình COKB

Mô hình COKB tổ chức tri thức theo sáu thành phần

$$\mathcal{K} = (C, H, R, \text{Ops}, \text{Funcs}, \text{Rules}). \quad (2.1)$$

Trong lý thuyết,  $C$  chứa các lớp đối tượng tính toán;  $H$  mô tả phân cấp;  $R$  là các quan hệ khác;  $\text{Ops}$  và  $\text{Funcs}$  mô tả phép toán và hàm;  $\text{Rules}$  chứa các luật suy diễn [1].

Đồ án vận dụng cách tổ chức này cho bài toán ánh xạ mã. Các đối tượng Python hiện chủ yếu là cấu trúc dữ liệu bất biến; các hành vi đánh giá được đặt trong operators, rules và reasoner. Vì vậy, đây là *một hiện thực chuyên biệt theo cấu trúc COKB*; chưa hiện thực đầy đủ cơ chế đối tượng tự giải, giải phương trình nội bộ, hay suy diễn đến bao đóng của một hệ COKB tổng quát.

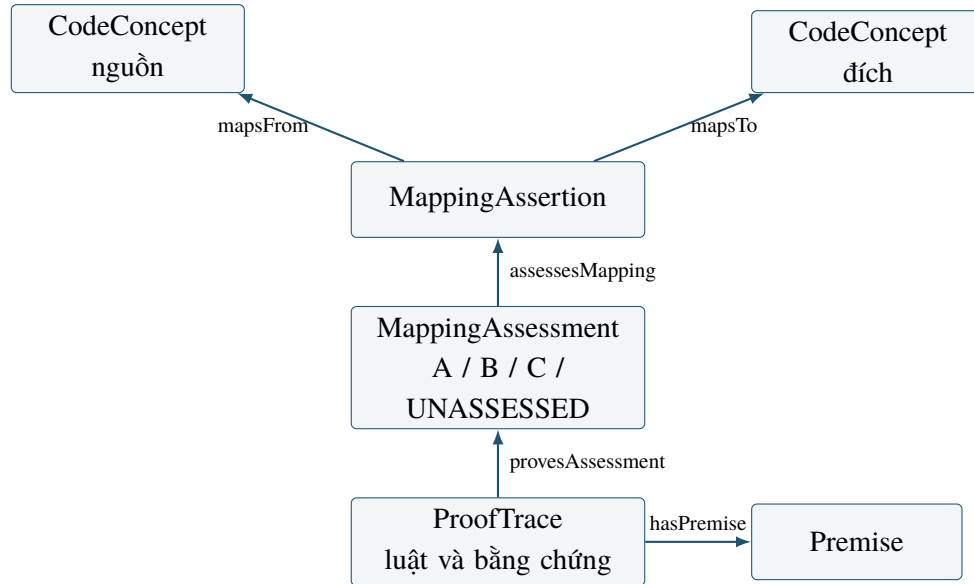
**Bảng 2.1:** Đối chiếu sáu thành phần COKB với source code

| Thành phần | Cụ thể hóa trong đồ án                                        | Hiện thực và giới hạn                                                |
|------------|---------------------------------------------------------------|----------------------------------------------------------------------|
| $C$        | CodeConcept, MappingAssertion, MappingAssessment, ProofTrace. | model.py và vocabulary Turtle; một số lớp chỉ có trong schema.       |
| $H$        | Phân cấp hệ mã, loại assessment, loại target.                 | rdfs:subClassOf; không phải cây phân cấp bệnh dùng để suy luận nhân. |
| $R$        | mapsFrom, mapsTo, assessesMapping, hasPremise.                | Quan hệ RDF do repository.py xuất.                                   |
| Ops        | Chuẩn hóa mã/nhân, tìm xung đột, so sánh tập từ.              | operators.py; khử trùng cặp trong importer.py.                       |
| Funcs      | Tìm mapping, đánh giá, giải thích, xuất kho tri thức.         | find_mappings, infer, explain, save.                                 |
| Rules      | Năm luật có ưu tiên và vết suy luận.                          | rules.py, reasoner.py; logic chạy bằng Python.                       |

Tên file Python trong bảng nằm dưới scripts/cokb/. Ontology nằm ở ontology/bio medical\_mapping\_cokb.ttl. Các cá thể Operator, Function và InferenceRule trong Turtle đóng vai trò danh mục tri thức; chúng không tự thực thi được. Danh mục rules/mapping\_rules.json cũng không phải ngôn ngữ luật mà reasoner đọc để chạy. Thứ tự thực thi thật được quyết định bởi DEFAULT\_RULES.

## 2.2 Các đối tượng và quan hệ

CodeConcept gồm hệ mã, mã và nhãn. MappingAssertion liên kết hai CodeConcept với bằng chứng nguồn. MappingAssessment bổ sung mức đánh giá, loại tác nhân, lời giải thích và proof. Premise chứa bộ ba thuộc tính predicate, value, source. Những thuộc tính này thể hiện cụ thể tri thức mà phép đánh giá được phép dùng.



**Hình 2.1:** Các đối tượng trung tâm và chiều quan hệ được xuất trong RDF.

Điểm đáng chú ý ở Hình 2.1 là mức A/B/C thuộc về *assessment của mapping*. Dữ kiện “mapping tồn tại” và kết luận “mapping được đánh giá mức A” có chủ thể và trách nhiệm khác nhau. Trong JSON, proof ghi mapping\_id cùng predicate và object; trong RDF, liên kết hasMappingLevel xuất phát từ assessment.

## 2.3 Phân cấp và mức độ hoàn thiện

Ontology khai báo BiomedicalCodeSystem là lớp con của CodeSystem; các lớp ICD nằm dưới BiomedicalCodeSystem. Các lớp RuleBasedAssessment, ModelAssessment, HumanAssessment nằm dưới MappingAssessment. Tuy nhiên, dữ liệu CodeConcept hiện ghi tên hệ mã bằng chuỗi codeSystem; chưa liên kết tới một cá thể CodeSystem được quản lý phiên bản.

Tương tự, SimpleTarget, OneOfTarget, AllOfTarget đã có trong vocabulary, nhưng MappingAssertion Python chỉ chứa một target. Không thể suy từ việc có tên lớp trong ontology rằng chương trình đã giải được lựa chọn “một trong nhiều mã” hoặc tổ hợp “phải dùng đồng thời nhiều mã”.

## 2.4 Dữ kiện nguồn, kết luận và bằng chứng

Kho tri thức được xuất thành ba file:

$$G_{\text{export}} = G_{\text{asserted}} \cup G_{\text{inferred}} \cup G_{\text{proof}}. \quad (2.2)$$

`asserted_graph.ttl` lưu cặp mã, nhãn, tác nhân và nguồn. `inferred_graph.ttl` lưu `assessment`, mức đánh giá và luật. `proof_graph.ttl` lưu các premise cùng liên kết tới `assessment`. Đây là ba tài liệu Turtle; lệnh xuất hiện tại không tự tạo ba named graph trong một RDF Dataset.

Quan hệ `prov:wasDerivedFrom` được dùng để liên kết mapping với `EvidenceSource`, phù hợp với mục đích mô tả nguồn gốc dữ liệu của PROV-O [4]. Ở bản cài đặt hiện tại, evidence là định danh chuỗi do fixture hoặc tham số import cung cấp. Chưa có cơ chế xác thực nội dung nguồn, snapshot nguồn, timestamp hay hash phiên bản luật trong proof. Do đó, truy vết cấu trúc đã có nhưng khả năng kiểm toán lịch sử còn hạn chế.

## 2.5 Ràng buộc tính hợp lệ

Kiểm tra Python yêu cầu đủ hệ mã, mã, nhãn nguồn/đích và evidence source; phát hiện trùng `mapping_id`; cảnh báo ánh xạ một mã tới chính nó. Kiểm tra SHACL áp dụng ràng buộc số lượng và kiểu liên kết trên dữ liệu RDF [3]. Ví dụ, một assertion phải có đúng một `mapsFrom`, đúng một `mapsTo` và ít nhất một `prov:wasDerivedFrom`.

Gọi  $V(M)$  là tập lỗi của một tập mapping  $M$ , trạng thái hợp lệ trong code là

$$\text{conforms}(M) \iff \nexists v \in V(M) : \text{severity}(v) = \text{error}. \quad (2.3)$$

Một warning không làm trạng thái này chuyển thành false. Tính hợp lệ về cấu trúc không chứng minh tính đúng của nhãn hay mapping. Chẳng hạn, một chuỗi evidence sai nội dung nhưng không rỗng vẫn có thể vượt qua kiểm tra hiện tại.

## Chương 3

# Toán tử và cơ chế suy luận

### 3.1 Chuẩn hóa và trích đặc trưng

Gọi  $N(\ell)$  là phép chuẩn hóa nhãn được cài đặt trong `normalize_label`. Hàm thực hiện lần lượt Unicode NFKC, chuyển dạng chữ bằng `casefold`, thay một tập dấu phân cách thành khoảng trắng, rút gọn khoảng trắng và thay từ theo bảng bí danh. Bảng bí danh hiện chỉ có ba mục: `haemorrhage` thành `hemorrhage`, `haemorrhagic` thành `hemorrhagic` và `renal` thành `kidney`. Đây là tri thức từ vựng được khai báo thủ công, không phải mô hình học đồng nghĩa.

Tập từ nội dung được định nghĩa theo code là

$$T(\ell) = \text{set}(\text{split}(N(\ell))) \setminus W, \quad W = \{a, an, and, due, of, or, the, to\}. \quad (3.1)$$

Biểu diễn tập hợp bỏ qua thứ tự và số lần xuất hiện của từ. Phép chuẩn hóa mã là loại khoảng trắng rồi viết hoa; dấu chấm trong mã được giữ. Tuy nhiên, chuẩn hóa mã chủ yếu phục vụ tìm kiếm và kiểm tra self-mapping, không được áp dụng thống nhất ở mọi bước tạo định danh.

### 3.2 Phát hiện xung đột theo cụm từ

Gọi  $D$  là tập các cặp cụm từ đối lập được khai báo. Một phần tử điển hình là `(acute, chronic)`; những cặp khác gồm `benign` và `malignant`; `left` và `right`; `type 1` và `type 2`; `with hemorrhage` và `without hemorrhage`; `hemorrhagic` và `without hemorrhage`; `with perforation` và `without perforation`. Phép kiểm tra có dạng

$$X(\ell_s, \ell_t) = \{(u, v) \in D : [u \sqsubseteq N(\ell_s) \wedge v \sqsubseteq N(\ell_t)] \vee [v \sqsubseteq N(\ell_s) \wedge u \sqsubseteq N(\ell_t)]\}, \quad (3.2)$$

trong đó  $\sqsubseteq$  là quan hệ xuất hiện như *chuỗi con*. Ký hiệu này không biểu diễn quan hệ chuyên biệt hóa bệnh. Điều kiện xung đột là  $X(\ell_s, \ell_t) \neq \emptyset$ .

Do dùng chuỗi con, phép kiểm tra chưa hiểu phạm vi phủ định hoặc cấu trúc “unspecified as acute or chronic”. Danh sách xung đột được lưu vào proof giúp người dùng phát hiện những nhận định quá đơn giản này.

### 3.3 Năm luật và thứ tự áp dụng

Đặt  $S = T(\ell_s)$ ,  $T = T(\ell_t)$  và  $U = \{\text{unspecified, other}\}$ . Các điều kiện  $P_j$  của năm luật được viết tương ứng với Python như sau:

$$P_1(m) : N(\ell_s) = N(\ell_t), \quad y_1 = A, \quad (3.3)$$

$$P_2(m) : X(\ell_s, \ell_t) \neq \emptyset, \quad y_2 = C, \quad (3.4)$$

$$P_3(m) : X = \emptyset \wedge S \neq \emptyset \wedge T \neq \emptyset \wedge (S \subsetneq T \vee T \subsetneq S), \quad y_3 = B, \quad (3.5)$$

$$P_4(m) : X = \emptyset \wedge S \cap T \neq \emptyset \wedge (S \cup T) \cap U \neq \emptyset, \quad y_4 = B, \quad (3.6)$$

$$P_5(m) : \text{true}, \quad y_5 = \perp. \quad (3.7)$$

Trong (3.5)–(3.6),  $X$  viết gọn cho  $X(\ell_s, \ell_t)$ . Luật thứ tư chỉ yêu cầu có từ chung và có từ chỉ tính khái quát; nó không kiểm tra rằng từ chung là tên bệnh hay rằng hai khái niệm thuộc cùng một nhánh ontology.

**Bảng 3.1:** Mã luật và bằng chứng được trả về khi luật khớp

| Ưu tiên | Mã luật                       | Premise trong proof                     |
|---------|-------------------------------|-----------------------------------------|
| 1       | R-EXACT-LABEL                 | Hai nhãn sau chuẩn hóa.                 |
| 2       | R-EXPLICIT-QUALIFIER-CONFLICT | Hai nhãn gốc và danh sách xung đột.     |
| 3       | R-COMPATIBLE-SPECIFICITY      | Hai nhãn gốc và explicitConflict=false. |
| 4       | R-GENERALIZED-CONDITION       | Hai nhãn gốc và explicitConflict=false. |
| 5       | R-INSUFFICIENT-EVIDENCE       | Hai nhãn gốc.                           |

`COKBReasoner.assess` dừng ở luật đầu tiên khớp:

$$j^*(m) = \min\{j : P_j(m) = \text{true}\}, \quad f_K(m) = (y_{j^*}, \text{rule\_based}, \pi_{j^*}). \quad (3.8)$$

Luật mặc định  $P_5$  bảo đảm luôn có đầu ra với tập luật chuẩn. Thứ tự là một phần ngữ nghĩa của hệ thống: nhãn giống hệt nhau được xét trước xung đột. Một tập luật tùy chỉnh không có fallback có thể làm reasoner phát sinh lỗi khi không luật nào khớp.

**Mã minh họa 3.1:** Giả mã tương ứng với vòng lặp của `COKBReasoner.assess`

```
def assess(mapping, ordered_rules):
    for rule in ordered_rules:
        match = rule.evaluate(mapping)
        if match is None:
            continue
        proof = build_proof(mapping, match)
        return build_assessment(mapping, match, proof)
    raise RuntimeError("Rule set did not produce an assessment")
```

Giả mã trong Mã minh họa 3.1 lược bỏ thao tác dựng dataclass. Đây là cơ chế áp dụng luật một lượt trên từng mapping, không lặp đến điểm cố định, không sử dụng assessment của mapping trước làm tiền đề cho mapping sau và không tự suy diễn bắc cầu giữa các hệ mã.

### 3.4 Vết suy luận và ý nghĩa của tính tái lập

Với một luật khớp, vết suy luận có cấu trúc

$$\pi = (q, \mathcal{P}, r, \mathcal{E}), \quad \frac{\mathcal{P}; r}{q}, \quad (3.9)$$

trong đó  $q$  là kết luận,  $\mathcal{P}$  là tập premise,  $r$  là mã luật và  $\mathcal{E}$  là nguồn bằng chứng. Ký hiệu phân số trong (3.9) mô tả cấu trúc giải trình, không khẳng định chương trình đã có một bộ kiểm chứng chứng minh độc lập.

Proof của luật 3 và 4 lưu nhãn nhưng chưa lưu tường minh tập từ sau xử lý, quan hệ bao hàm hay từ chung. Proof của fallback cũng chưa liệt kê vì sao từng luật trước thất bại. Do đó, vết hiện tại đủ để xác định luật và dữ liệu đầu vào, nhưng chưa là nhật ký đầy đủ của mọi bước tính toán.

Tính xác định được hiểu là cùng input, cùng code và cùng thứ tự luật thì thu được cùng assessment. Trong `COKBRepository.load`, nếu JSON đã có assessments, code tính lại chúng bằng luật hiện hành. Vì thế, mở một store sau khi sửa luật có thể cho kết quả mới; muốn tái lập lịch sử cần lưu phiên bản code bên cạnh dữ liệu.

### 3.5 Độ phức tạp và giới hạn biểu diễn

Gọi  $n$  là số mapping,  $k$  là số luật và  $L$  là tổng độ dài hai nhãn. Với số cặp xung đột cố định, mỗi phép chuẩn hóa và kiểm tra có chi phí xấp xỉ tuyến tính theo  $L$ . Tổng chi phí đánh giá có dạng

$$O(nkL), \quad k = 5 \text{ trong tập luật mặc định.} \quad (3.10)$$

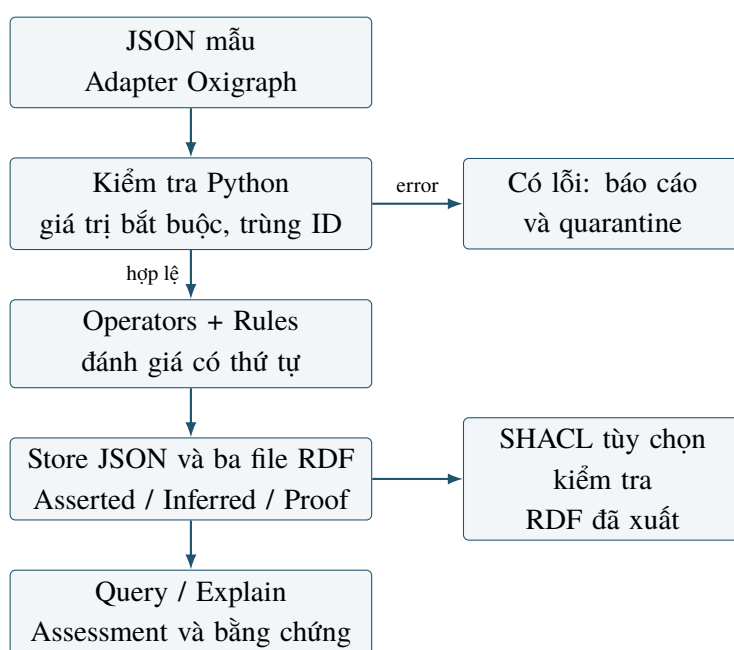
Một số phép chuẩn hóa được lặp lại qua các luật; chưa có cache đặc trưng. Truy vấn `find_mappings` quét danh sách mapping nên có chi phí  $O(n)$  nếu xem chiều dài mã là bị chặn. Các công thức này là phân tích thuật toán từ source, không phải kết quả đo tốc độ hay cam kết hiệu năng trên graph đầy đủ.

## Chương 4

# Thiết kế và cài đặt hệ thống

### 4.1 Kiến trúc xử lý

Lớp COKB nhận dữ liệu JSON chuẩn hóa hoặc dữ liệu chuyển đổi từ Oxigraph. Với lệnh `build/reason/import`, CLI kiểm tra cấu trúc trước, chỉ suy luận khi không có lỗi mức `error`, sau đó xuất store cùng các graph và proof. SHACL được gọi bằng lệnh riêng sau khi xuất RDF.



**Hình 4.1:** Luồng COKB được thực thi bởi các lệnh CLI hiện tại.

### 4.2 Phân rã module

Lỗi suy luận và đọc XLSX có thể chạy bằng thư viện chuẩn Python. Theo `pyproject.toml`, môi trường project yêu cầu Python từ 3.11 đến trước 3.13; các phụ thuộc tùy chọn nhóm `validation` là `rdflib` và `pyshacl`. Lệnh `import` cần `pyoxigraph`. Các chức năng COKB được gọi từ `main.py` mà không phải cấu hình LLM.



**Bảng 4.1:** Trách nhiệm các module của lớp COKB

| Module trong <code>scripts/</code><br><code>cokb/</code> | Trách nhiệm                                                   |
|----------------------------------------------------------|---------------------------------------------------------------|
| <code>model.py</code>                                    | Dataclass, enum mức đánh giá, ID ổn định, validation report.  |
| <code>operators.py</code>                                | Chuẩn hóa, tập từ, xung đột, điều kiện so sánh.               |
| <code>rules.py, reasoner.py</code>                       | Điều kiện luật, chọn luật, dựng assessment và proof.          |
| <code>validation.py, shacl.py</code>                     | Kiểm tra Python và kiểm tra RDF bằng pySHACL.                 |
| <code>repository.py</code>                               | Nạp/lưu, truy vấn, giải thích, tuần tự hóa RDF và JSON.       |
| <code>importer.py</code>                                 | Đọc Oxigraph ở chế độ read-only và chuyển cặp mã.             |
| <code>xlsx.py, evaluation.py</code>                      | Đọc sheet gold bằng thư viện chuẩn, tính chỉ số đánh giá.     |
| <code>cli.py</code>                                      | Khai báo subcommand, kiểm soát luồng, mã thoát, xuất báo cáo. |

### 4.3 Định danh và nhập dữ liệu

Nếu input chưa cung cấp ID, `stable_mapping_id` băm chuỗi nối hệ mã, mã nguồn, hệ mã và mã đích bằng SHA-256, lấy 16 ký tự hex đầu:

$$id(m) = \text{mapping-} \parallel \text{prefix}_{16}(\text{SHA256}(s_s \parallel k_s \parallel s_t \parallel k_t)). \quad (4.1)$$

Định danh này bỏ qua label và evidence. Hai mapping có cùng bốn thành phần nhưng nguồn khác nhau có thể cùng ID; đây là giới hạn thiết kế, không phải cơ chế quản lý lịch sử nhiều bằng chứng.

Adapter dùng SPARQL `SELECT DISTINCT` trong các named graph để lấy cặp mã và nhãn, rồi khử trùng theo bộ  $(s_s, k_s, s_t, k_t)$ . Các tham số `source-system`, `target-system` và `evidence-source` được truyền từ CLI, không được suy ra riêng cho từng dòng. Vì vậy, nếu nguồn chứa lẫn ICD10WHO và ICD10CM thì việc gán một hệ nguồn cho toàn bộ kết quả có thể làm sai metadata. Code cũng chưa bảo toàn graph IRI và mapping IRI của từng dòng làm provenance.

### 4.4 Xử lý lỗi và các artifact

Khi validation phát hiện error, CLI ghi `validation_report.json` và `quarantine.json`, trả mã thoát 1 và không tiếp tục inference của lần gọi đó. Cả batch bị dừng; không có cơ chế tự động xử lý phần hợp lệ còn lại. API `infer` và `explain` không tự bắt buộc validation, nên ứng dụng tích hợp phải gọi kiểm tra rõ ràng.

Khi thành công, save tạo sáu artifact: `store.json`, `validation_report.json`, `proofs.json`, `asserted_graph.ttl`, `inferred_graph.ttl`, `proof_graph.ttl`. SHACL hợp

nhất ba file RDF cùng ontology để kiểm tra; tùy chọn suy luận RDFS trong bước này phục vụ nhận biết lớp và ràng buộc, không thay thế bộ đánh giá A/B/C bằng Python.

### Vị trí bổ sung hình minh họa

Chụp terminal sau khi chạy `cokb_build` và `cokb_query`. Hiển thị hai target DA0B.Y và DA0Z, mức A và UNASSESSED, cùng mã luật của mỗi assessment. Chỉ dùng output thật của repo.

Tệp dự kiến: `images/cli_demo.png`

**Hình 4.2:** Kết quả truy vấn COKB cho mã nguồn K05.1.

## 4.5 Ví dụ giải thích một mapping

Với cặp K05.1 và DA0B.Y trong fixture, hai nhãn đều là *Chronic gingivitis*. Sau chuẩn hóa, cả hai thành `chronic gingivitis`. Luật đầu tiên khớp, trả A với hai premise là nhãn đã chuẩn hóa và nguồn do fixture khai báo.

**Mã minh họa 4.1:** Trích phần proof của ví dụ nhãn trùng nhau

```
{
  "conclusion": {
    "mapping_id": "mapping-k05-1-da0b-y",
    "predicate": "hasMappingLevel",
    "object": "A"
  },
  "rule_id": "R-EXACT-LABEL",
  "sources": ["icd10cm_to_icd11_2024_full"]
}
```

Mã minh họa 4.1 lược bỏ mảng premises cho gọn. Chuỗi source trong fixture là metadata tự khai báo; tên của nó không chứng minh ví dụ đã được xác minh với toàn bộ file ICD.

### Vị trí bổ sung hình minh họa

Nạp ba file Turtle và ontology vào công cụ xem RDF. Vẽ các node mapping, assessment mức A, R-EXACT-LABEL, hai premise nhãn chuẩn hóa và evidence; ghi tên cạnh theo RDF đã xuất. Ảnh cần cho thấy đường đi từ assessment đến dữ kiện hỗ trợ.

Tệp dự kiến: `images/proof_graph.png`

**Hình 4.3:** Trực quan hóa vết suy luận cho mapping K05.1 đến DA0B.Y.

## Chương 5

# Thực nghiệm và phân tích kết quả

### 5.1 Thiết kế thực nghiệm

Báo cáo kiểm tra ba câu hỏi: hệ thống có xuất được assessment và proof cho dữ liệu mẫu hay không; năm luật bao phủ bao nhiêu cặp nhãn gold; và độ chính xác trong phần được đánh giá là bao nhiêu. Các kết quả dưới đây được chạy lại từ source ngày 07/09/2026 bằng Python 3.12.14 trong môi trường `.venv`.

**Bảng 5.1:** Các dữ liệu và kiểm tra được sử dụng

| Thành phần   | Quy mô       | Mục đích và giới hạn                                                       |
|--------------|--------------|----------------------------------------------------------------------------|
| Fixture JSON | 5 mapping    | Minh họa chức năng build, query, explain và bốn mức đầu ra.                |
| Gold XLSX    | 500 cặp nhãn | Đo khả năng phân loại A/B/C; không đo khả năng tìm mã đích.                |
| Unit tests   | 19 test      | Kiểm tra các nhánh code; SHACL là một test tùy chọn.                       |
| Graph store  | 0 quad       | Trạng thái đọc tại lần kiểm tra; chưa cung cấp dữ liệu thực nghiệm đầy đủ. |
| TTL nguồn    | LFS pointer  | Khai báo object 262.786.162 byte, chưa phải nội dung RDF đầy đủ.           |

Gold nằm tại `gold_datasets/mapping_level/code_pair_mapping_level_gold_standard__reasoning.xlsx`. Evaluator đọc hai nhãn và nhãn chuẩn từ sheet đầu tiên, dựng CodeConcept với mã giả S0/T0, S1/T1, rồi gọi reasoner. Trường gold chỉ dùng để so sánh sau dự đoán, không được đưa vào điều kiện luật. Tên evidence `mapping_level_gold_standard` chỉ là metadata của fixture đánh giá. Tập dữ liệu này là bộ tham chiếu sẵn có; không có thiết kế chia train/validation/test mới hay khảo sát chuyên gia độc lập trong đề án.

## 5.2 Các chỉ số

Gọi  $N$  là số cặp,  $g_i \in \{A, B, C\}$  là nhãn chuẩn,  $\hat{y}_i \in \{A, B, C, \perp\}$  là dự đoán. Đặt  $a_i = \mathbf{1}[\hat{y}_i \neq \perp]$  và  $c_i = \mathbf{1}[\hat{y}_i = g_i]$ . Khi đó

$$\text{Coverage} = \frac{\sum_i a_i}{N}, \quad (5.1)$$

$$\text{Accuracy}_{\text{all}} = \frac{\sum_i c_i}{N}, \quad (5.2)$$

$$\text{Accuracy}_{\text{assessed}} = \frac{\sum_i c_i}{\sum_i a_i}, \quad (5.3)$$

$$\text{Abstention} = 1 - \text{Coverage}. \quad (5.4)$$

Trong code, tỷ số có mẫu số bằng 0 được trả về 0. Với tập gold chỉ gồm A/B/C, có đẳng thức

$$\text{Accuracy}_{\text{all}} = \text{Coverage} \cdot \text{Accuracy}_{\text{assessed}}. \quad (5.5)$$

Việc coi UNASSESSED là không đúng trong accuracy toàn tập không biến nó thành một lớp bệnh hay một nhãn gold mới.

Với từng mức  $\ell \in \{A, B, C\}$ ,

$$P_\ell = \frac{TP_\ell}{TP_\ell + FP_\ell}, \quad R_\ell = \frac{TP_\ell}{TP_\ell + FN_\ell}, \quad F_{1,\ell} = \frac{2P_\ell R_\ell}{P_\ell + R_\ell}. \quad (5.6)$$

Các cặp gold mức  $\ell$  nhưng bị từ chối vẫn nằm trong  $FN_\ell$ . Do đó recall phản ánh cả giới hạn bao phủ.

## 5.3 Kết quả chức năng trên năm mapping

**Bảng 5.2:** Đầu ra tái lập trên fixture có sẵn

| Nguồn | Đích   | Đầu ra  | Luật được chọn               |
|-------|--------|---------|------------------------------|
| K05.1 | DA0B.Y | A       | Exact label.                 |
| K05.1 | DA0Z   | $\perp$ | Insufficient evidence.       |
| K25.9 | DA60.Y | C       | Explicit qualifier conflict. |
| A31   | 1B21.Z | A       | Exact label.                 |
| A31   | 1H0Z   | B       | Generalized condition.       |

Kiểm tra Python của fixture trả về conforms=true, không có error và warning. Năm kết quả có đủ conclusion, premise, rule ID và source trong cấu trúc proof. Đây là kiểm tra chức năng trên ví dụ minh họa; không diễn giải thành độ chính xác 100% trên một bộ kiểm thử y sinh độc lập.

Với K05.1 đến DA0Z, hai tập từ không có phần giao phù hợp để kích hoạt luật; hệ thống trả  $\perp$  dù nhãn đích chứa “unspecified”. Điều này minh họa rằng có tên mã và có từ chỉ tính khái quát chưa đủ để reasoner suy ra mức B.

Trong ví dụ K25.9 đến DA60.Y, cụm “without hemorrhage” xuất hiện ở nhãn nguồn; nhãn đích có từ “haemorrhagic”. Phép chuẩn hóa và tìm cụm từ kích hoạt luật xung đột. Tuy nhiên, luật cũng bắt gặp “chronic” trong “unspecified as acute or chronic” và có thể đưa vào danh sách acute/chronic. Trường hợp này cho thấy một đầu ra C có thể đi cùng một premise phụ chưa được diễn giải ngữ nghĩa chính xác.

## 5.4 Kết quả trên bộ gold

**Bảng 5.3:** Kết quả tổng hợp của bộ suy luận xác định

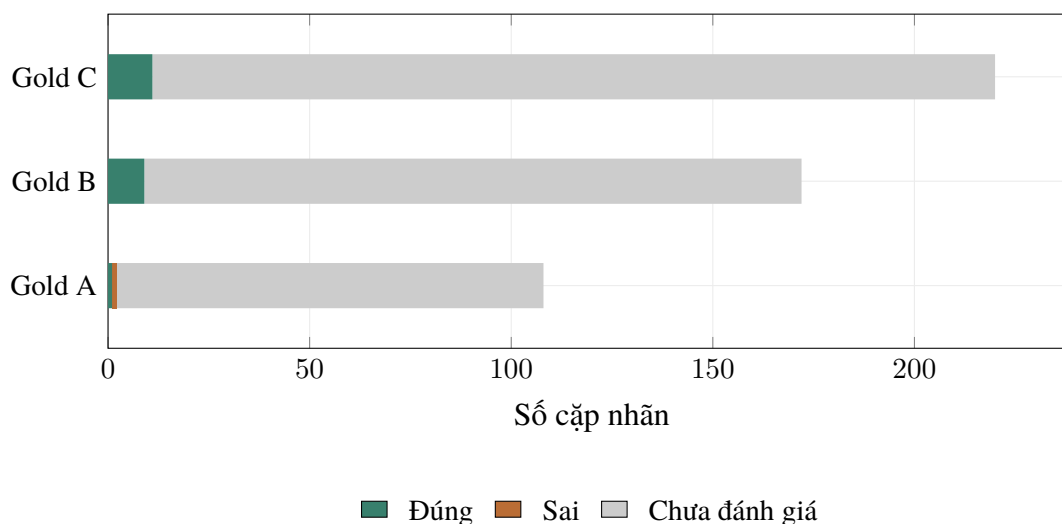
| Chỉ số                        | Phân số hoặc số lượng | Giá trị |
|-------------------------------|-----------------------|---------|
| Tổng số cặp                   | 500                   | 100,00% |
| Được đánh giá                 | 22/500                | 4,40%   |
| Đúng trên toàn bộ tập         | 21/500                | 4,20%   |
| Đúng trong phần được đánh giá | 21/22                 | 95,45%  |
| Sai trong phần được đánh giá  | 1/22                  | 4,55%   |
| Chưa được đánh giá            | 478/500               | 95,60%  |

**Bảng 5.4:** Ma trận nhầm lẫn: hàng là gold, cột là dự đoán

| Gold | A | B  | C  | $\perp$ | Tổng |
|------|---|----|----|---------|------|
| A    | 1 | 1  | 0  | 106     | 108  |
| B    | 0 | 9  | 0  | 163     | 172  |
| C    | 0 | 0  | 11 | 209     | 220  |
| Tổng | 1 | 10 | 11 | 478     | 500  |

**Bảng 5.5:** Precision, recall và F1 theo từng mức; đơn vị %

| Mức | Precision | Recall | F1   |
|-----|-----------|--------|------|
| A   | 100,00    | 0,93   | 1,83 |
| B   | 90,00     | 5,23   | 9,89 |
| C   | 100,00    | 5,00   | 9,52 |



**Hình 5.1:** Phân bố kết quả theo nhãn chuẩn. Phần chưa đánh giá chiếm ưu thế ở cả ba nhóm; các số đếm chi tiết nằm trong Bảng 5.4.

Các bảng và Hình 5.1 thể hiện hai góc nhìn khác nhau: phần được xử lý có ít lỗi, nhưng phần lớn dữ liệu chưa được xử lý. Con số 95,45% chỉ dựa trên 22 cặp; không thể dùng riêng nó để kết luận hệ thống đạt hiệu quả cao trên toàn bộ miền bài toán. Precision 100% của A cũng chỉ dựa trên một dự đoán A.

## 5.5 Mức đóng góp của từng luật

**Bảng 5.6:** Số lần từng luật được chọn trên 500 cặp gold

| Luật                        | Số lần được chọn | Số dự đoán đúng A/B/C |
|-----------------------------|------------------|-----------------------|
| Exact label                 | 1                | 1                     |
| Explicit qualifier conflict | 11               | 11                    |
| Compatible specificity      | 10               | 9                     |
| Generalized condition       | 0                | 0                     |
| Insufficient evidence       | 478              | Không áp dụng         |

Các số này thống kê *luật được chọn đầu tiên*, không phải số lần điều kiện của mỗi luật đúng nếu kiểm tra độc lập. Luật generalized có ví dụ dương trong fixture nhưng không được chọn trên bộ gold. Đồ án chưa thực hiện ablation thay đổi thứ tự hoặc bỏ từng luật, nên chưa kết luận được đóng góp nhân quả của mỗi luật tới chất lượng.

## 5.6 Phân tích lỗi cụ thể

Cặp gold có trường `Label=378` là:

Nguồn: Diabetes mellitus type 2.

Đích: Type 2 diabetes.

Gold: A. Dự đoán: B.

Hai tập từ sau chuẩn hóa là

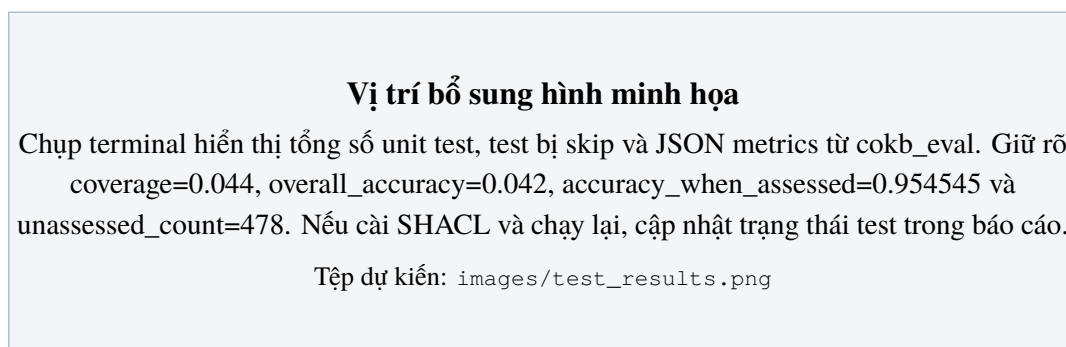
$$S = \{\text{diabetes,mellitus,type,2}\}, \quad T = \{\text{type,2,diabetes}\}, \quad T \subsetneq S. \quad (5.7)$$

Chuỗi không giống hệt nhau nên luật 1 không khớp; không có xung đột nên luật 3 trả B. Theo gold, khác biệt từ “mellitus” ở cặp này không làm thay đổi mức tương đương. Lỗi phản ánh khoảng cách giữa bao hàm từ vựng và quan hệ ngữ nghĩa.

Hướng cải thiện là thêm tri thức đồng nghĩa theo miền, có kiểm thử âm và đánh giá lại độ bao phủ. Không nên xóa một từ trên mọi nhãn chỉ để sửa đúng một dòng gold, vì có thể tạo ra lỗi ở những trường hợp chưa quan sát.

## 5.7 Kiểm thử và tính tái lập

Lần chạy `python-munittestdiscover-stests-v` thu được 19 test, gồm 18 đạt và 1 bị bỏ qua do thiếu dependency SHACL tùy chọn. Các nhóm đã chạy gồm operators, năm nhánh reasoner, repository, validation, importer với dữ liệu giả lập, XLSX, evaluation và SPARQL guard. Không báo cáo rằng SHACL đã chạy thành công trong môi trường này.



**Hình 5.2:** Minh họa kết quả kiểm thử và đánh giá tái lập.

Script kèm báo cáo lưu JSON đánh giá, phiên bản Python, hash các file source liên quan, hash dữ liệu gold, kết quả unit test và artifact mẫu. Các số trong bảng được đối chiếu với những artifact này. Không có số đo latency, chi phí LLM, hiệu quả người dùng hoặc kết quả triển khai trên toàn bộ ICD trong lần thực nghiệm này.

## Chương 6

# Thảo luận, kết luận và hướng phát triển

### 6.1 Đóng góp của đề án

Đề án cụ thể hóa bài toán ánh xạ mã thành các đối tượng, quan hệ, phép xử lý và luật có thể thực thi. Điều quan trọng đối với môn Biểu diễn tri thức là mối liên hệ giữa ba phần: tri thức được lưu dưới dạng gì, bộ suy luận sử dụng phần tri thức nào và người dùng kiểm tra kết quả bằng cách nào. Các module và công thức trong báo cáo thể hiện trực tiếp mối liên hệ đó.

Kết quả thứ nhất là hợp đồng dữ liệu phân biệt assertion, assessment và proof. Kết quả thứ hai là bộ suy luận có thứ tự, có đầu ra từ chối rõ ràng. Kết quả thứ ba là bộ thực nghiệm vừa báo cáo độ chính xác, vừa thể hiện phần tri thức chưa được bao phủ. Những kết quả này giúp việc phát triển thêm luật có điểm xuất phát đo được.

### 6.2 Ý nghĩa trong một quy trình sử dụng

Trong một quy trình rà soát mapping, hệ thống có thể hỗ trợ người dùng nhóm các cặp theo assessment và mở bằng chứng để kiểm tra. Một chính sách nghiệp vụ tương lai có thể được mô hình hóa là

$$d(m) = \text{Policy}(f_{\kappa}(m), \text{context}(m)). \quad (6.1)$$

Ở đây  $d$  là quyết định phê duyệt hoặc yêu cầu xem xét. `Policy` và `context` chưa được cài đặt trong repo. Việc tách này ngăn diễn giải sai rằng một mức A/B/C tự động tương đương với quyết định nghiệp vụ cuối cùng.

Proof trace cho phép kiểm tra và phát hiện sai sót ở từng luật. `UNASSESSED` tạo ra danh sách cụ thể các ca cần bổ sung tri thức. Đây là cơ sở để xây dựng công cụ hỗ trợ chuyên gia và theo dõi chất lượng dữ liệu. Khả năng giảm thời gian review hoặc giảm chi phí cần được đánh giá bằng một quy trình có người dùng và tiêu chí chấp nhận xác định.

### 6.3 Giới hạn cần nhìn nhận

Một hạn chế bổ sung là ontology và rule catalog chưa được dùng làm nguồn thực thi thống nhất. Thay đổi Turtle hoặc JSON catalog mà không sửa Python có thể làm tài liệu và hành vi khác nhau. Tính nhất quán giữa các biểu diễn cần được kiểm tra khi mở rộng.

Graph đầy đủ chưa có trong checkout. Indexer có cơ chế nạp dữ liệu mẫu nếu không đọc được TTL thực, nên việc một lệnh indexing hoàn thành không đủ chứng minh rằng toàn bộ dữ



**Bảng 6.1:** Khoảng cách giữa hiện thực hiện tại và hệ thống hoàn chỉnh

| Khía cạnh        | Hiện tại                                          | Phần cần bổ sung                                                                |
|------------------|---------------------------------------------------|---------------------------------------------------------------------------------|
| Ngữ nghĩa        | Chuỗi, bí danh nhỏ, tập từ, cặp từ đối lập.       | Đồng nghĩa có nguồn, phủ định, quan hệ phân cấp có kiểm chứng.                  |
| COKB             | Sáu nhóm thành phần được vận dụng trong miền hẹp. | Đối tượng có hành vi nội tại và bộ giải tổng quát nếu mục tiêu mở rộng đòi hỏi. |
| Provenance       | ID nguồn và mã luật.                              | Phiên bản nguồn, hash luật, thời điểm, tác nhân và lịch sử phê duyệt.           |
| Mapping phức hợp | Target đơn trong Python.                          | Ngữ nghĩa OneOf/AllOf và kiểm thử tương ứng.                                    |
| Đánh giá         | 500 cặp gold, coverage thấp.                      | Bộ holdout độc lập, ablation, kiểm định với chuyên gia.                         |
| Triển khai       | CLI và file JSON/Turtle.                          | API, quản lý người dùng, policy và quy trình review.                            |

liệu ICD đã được nạp. Trong lần kiểm tra dùng cho báo cáo, store có 0 quad. Cần kiểm tra số lượng, hệ mã và provenance sau import trước khi thực nghiệm quy mô lớn.

## 6.4 Hướng phát triển ưu tiên

Ưu tiên thứ nhất là nâng chất lượng tri thức nhân: bổ sung xử lý phủ định và đồng nghĩa có kiểm thử hồi quy, sau đó đánh giá trên tập độc lập. Ưu tiên thứ hai là hoàn thiện provenance và proof: lưu phiên bản nguồn, hash ruleset, các tập từ đã tính và lý do các luật không khớp. Ưu tiên thứ ba là phục hồi dữ liệu graph đầy đủ, xử lý đúng hệ mã của từng dòng và báo cáo số lượng mapping thực sự đã nạp.

Sau các bước đó có thể thử tích hợp lớp giao tiếp ngôn ngữ tự nhiên. Kết quả do model tạo cần có loại assessment riêng và được đánh giá đối chiếu với luật; hiện chưa có thực nghiệm hybrid trong đồ án. Nếu tiếp tục theo COKB tổng quát, cần thiết kể rõ tập loại sự kiện, hành vi đối tượng và chiến lược suy diễn, thay vì chỉ bổ sung thêm tên class vào ontology.

## 6.5 Kết luận

Đồ án cho thấy cách đưa tri thức kiểm tra mapping vào một cấu trúc có thể vận hành: đối tượng và quan hệ để biểu diễn, toán tử và luật để đánh giá, proof để giải thích, validation để kiểm tra dữ liệu. Thực nghiệm xác nhận các thành phần này hoạt động trên phạm vi hiện có, đồng thời chỉ ra giới hạn bao phủ 4,40%. Ý nghĩa thực tế của COKB ở đây là cung cấp những nhận định có thể kiểm tra và nhận diện rõ các ca chưa đủ tri thức, làm nền cho quy trình rà soát mapping có sự tham gia của chuyên gia.

## Phụ lục A

# Hướng dẫn tái lập và đối chiếu source

### A.1 Các lệnh tái lập

Thực hiện từ thư mục gốc repository. Các lệnh dưới đây dùng thư mục `reports/artifacts/demo` cho kết quả mẫu để phân biệt với graph store. Nếu sử dụng virtual environment, thay `python3` bằng `.venv/bin/python`.

```
python3 main.py ckb_build \
  --input examples/icd_mapping_sample.json \
  --output reports/artifacts/demo

python3 main.py ckb_query \
  --store reports/artifacts/demo \
  --code K05.1 --system ICD10WHO

python3 main.py ckb_explain \
  --store reports/artifacts/demo \
  --mapping-id mapping-k05-1-da0b-y

python3 main.py ckb_eval \
  --output reports/artifacts/evaluation.json

python3 -m unittest discover -s tests -v
```

Script `reports/scripts/collect_evidence.py` thực hiện thu thập kết quả dùng trong báo cáo bằng một lệnh:

```
python3 reports/scripts/collect_evidence.py
```

Script chỉ ghi vào `reports/artifacts/`; không thay đổi ontology, rules, gold hoặc graph store gốc.

SHACL là kiểm tra bổ sung sau khi cài dependency:

```
python3 -m pip install -e '.[validation]'
python3 main.py ckb_shacl \
  --store reports/artifacts/demo \
  --report reports/artifacts/shacl_report.ttl
```

Lệnh này phải được chạy và ghi nhận riêng trước khi thay trạng thái SHACL trong kết quả thực nghiệm.

## A.2 Chỉ mục đối chiếu nội dung với code

**Bảng A.1:** Các điểm kiểm chứng chính trong repository

| Nội dung báo cáo                | Source và điểm kiểm tra                                                                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Đối tượng và mức đánh giá       | <code>scripts/cokb/model.py</code> : <code>CodeConcept</code> , <code>MappingAssertion</code> , <code>MappingAssessment</code> , <code>MappingLevel</code> .                 |
| Chuẩn hóa và xung đột           | <code>scripts/cokb/operators.py</code> : <code>normalize_label</code> , <code>content_tokens</code> , <code>find_explicit_conflicts</code> .                                 |
| Năm điều kiện luật              | <code>scripts/cokb/rules.py</code> : các lớp <code>Rule</code> và <code>DEFAULT_RULES</code> .                                                                               |
| Luật đầu tiên khớp              | <code>scripts/cokb/reasoner.py</code> : <code>COKBReasoner.assess</code> .                                                                                                   |
| Ba graph và tính lại assessment | <code>scripts/cokb/repository.py</code> : <code>load</code> , <code>infer</code> , <code>asserted_turtle</code> , <code>inferred_turtle</code> , <code>proof_turtle</code> . |
| Lỗi và cách ly batch            | <code>scripts/cokb/cli.py</code> : <code>run_cokb_command</code> , <code>_write_quarantine</code> .                                                                          |
| Các ràng buộc Python            | <code>scripts/cokb/validation.py</code> : <code>validate_mappings</code> .                                                                                                   |
| Các ràng buộc SHACL             | <code>shapes/biomedical_mapping_shapes.ttl</code> và <code>scripts/cokb/shacl.py</code> .                                                                                    |
| Import và khử trùng             | <code>scripts/cokb/importer.py</code> : <code>MAPPING_QUERY</code> , <code>repository_from_solutions</code> .                                                                |
| Độ bao phủ và accuracy          | <code>scripts/cokb/evaluation.py</code> : <code>evaluate_label_pairs</code> .                                                                                                |
| Cơ chế dữ liệu mẫu thay thế     | <code>scripts/utils/indexer.py</code> : <code>build_graph_store</code> , <code>is_git_lfs_pointer</code> .                                                                   |

## A.3 Danh sách hình cần bổ sung

Các sơ đồ đối tượng và kiến trúc cùng biểu đồ thực nghiệm được vẽ trực tiếp bằng TikZ/PGFPlots, không cần ảnh ngoài. Ba vị trí sau tự hiển thị ảnh khi file xuất hiện:

- Hình 4.2: `images/cli_demo.png`, ảnh truy vấn K05.1 cho thấy hai candidate và assessment.
- Hình 4.3: `images/proof_graph.png`, ảnh graph giải thích kết luận A với rule và premise.
- Hình 5.2: `images/test_results.png`, ảnh terminal unit test và metrics được chạy thật.

Mô tả ngay trong từng khung giữ chỗ nêu các đối tượng cần xuất hiện. LaTeX vẫn biên dịch được khi chưa có ba ảnh này.

## Tài liệu tham khảo

- [1] Nhon V. Do, Hien D. Nguyen và Thanh T. Mai. *Reasoning Method on Knowledge about Functions and Operators*. International Journal of Advanced Computer Science and Applications, 6(6), tr. 156–168, 2015. Mục III.A trình bày cấu trúc sáu thành phần COKB. [https://thesai.org/Downloads/Volume6No6/Paper\\_22-Reasoning\\_Method\\_on\\_Knowledge\\_about\\_Functions\\_and\\_Operators.pdf](https://thesai.org/Downloads/Volume6No6/Paper_22-Reasoning_Method_on_Knowledge_about_Functions_and_Operators.pdf).
- [2] Hui Feng, Yuntzu Yin, Emiliano Reynares và Jay Nanavati. *OntologyRAG: Better and Faster Biomedical Code Mapping with Retrieval-Augmented Generation (RAG) Leveraging Ontology Knowledge Graphs and Large Language Models*. arXiv:2502.18992, 2025. <https://arxiv.org/abs/2502.18992>.
- [3] Holger Knublauch và Dimitris Kontokostas (biên tập). *Shapes Constraint Language (SHACL)*. W3C Recommendation, 20/07/2017. <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [4] Timothy Lebo, Satya Sahoo và Deborah McGuinness (biên tập). *PROV-O: The PROV Ontology*. W3C Recommendation, 30/04/2013. <https://www.w3.org/TR/prov-o/>.